
Parte 5: La Interfaz de Usuario (UI Reactiva)

1. El Concepto: Consumo de Providers Asíncronos

Utilizaremos el método `.when()` de Riverpod. Esta es la forma más profesional de manejar interfaces en Flutter, ya que nos obliga a definir qué mostrar mientras los datos cargan o si ocurre un error.

2. Implementación: Pantalla de la Tienda (`screens/tienda_screen.dart`)

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../providers/catalogo_provider.dart';
import '../providers/carrito_provider.dart';

class TiendaScreen extends ConsumerWidget {
  const TiendaScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // 1. Escuchamos los tres estados que necesitamos
    final catalogoAsync = ref.watch(catalogoProvider);
    final carritoAsync = ref.watch(carritoProvider);
    final total = ref.watch(carritoTotalProvider);

    return Scaffold(
      appBar: AppBar(
        title: const Text('Ingeniería Store'),
        actions: [
          // Badge con el total de la compra
          Padding(
            padding: const EdgeInsets.all(16.0),
            child: Center(child: Text('Total: \${total.toStringAsFixed(2)}',
              style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 18))),
          ),
        ],
      ),
      body: Column(
        children: [
          const Padding(
            padding: EdgeInsets.all(8.0),
            child: Text('Catálogo de Productos', style: TextStyle(fontSize: 20,
              fontWeight: FontWeight.bold)),
          ),

          // SECCIÓN 1: EL CATÁLOGO (LECTURA)
          Expanded(
            flex: 2,
            child: catalogoAsync.when(
```

```

        data: (productos) => ListView.builder(
          itemCount: productos.length,
          itemBuilder: (context, i) => ListTile(
            title: Text(productos[i].nombre),
            subtitle: Text('\${productos[i].precio}'),
            trailing: IconButton(
              icon: const Icon(Icons.add_shopping_cart, color:
Colors.green),
              onPressed: () =>
ref.read(carritoProvider.notifier).agregarProducto(productos[i]),
            ),
          ),
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, st) => Center(child: Text('Error: $e')),
      ),
    ),

    const Divider(thickness: 2),

    const Padding(
      padding: EdgeInsets.all(8.0),
      child: Text('Tu Carrito (Persistente en Isar)', style:
TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),
    ),

    // SECCIÓN 2: EL CARRITO (LECTURA Y ACCIÓN)
    Expanded(
      flex: 1,
      child: carritoAsync.when(
        data: (items) => items.isEmpty
          ? const Center(child: Text('El carrito está vacío'))
          : ListView.builder(
              itemCount: items.length,
              itemBuilder: (context, i) => ListTile(
                leading: CircleAvatar(child: Text('${items[i].cantidad}')),
                title: Text(items[i].producto.value?.nombre ??
'Cargando...'),
                trailing: IconButton(
                  icon: const Icon(Icons.delete, color: Colors.red),
                  onPressed: () =>
ref.read(carritoProvider.notifier).eliminarItem(items[i].id),
                ),
              ),
            ),
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, st) => const SizedBox(),
      ),
    ),
  ],
),
);

```

```
}  
}
```

3. Teoría detrás de la UI: Optimización de Renders

- **Independencia de estados:** Gracias a que dividimos los providers, si agregas un producto al carrito, la sección del **Catálogo** NO se reconstruye (re-render), solo se reconstruye la sección del **Carrito** y el **Total** en la AppBar.
- **Feedback Inmediato:** Aunque Isar es muy rápido, Riverpod asegura que la UI se actualice en el microsegundo que la transacción termina.
- **Persistencia Visual:** Prueba lo siguiente:
 1. Agregar 3 monitores al carrito.
 2. Detener la aplicación por completo (Stop).
 3. Abrirla de nuevo.
 4. *Magia:* Los 3 monitores siguen ahí porque `build()` en el `CarritoNotifier` volvió a consultar a Isar.

4. Resumen del Flujo de Ingeniería

El camino que se recorrió:

1. **Definición:** Creamos clases con `@collection` (Modelos).
 2. **Persistencia:** Inicializamos `Isar` y creamos un seed de datos (Infraestructura).
 3. **Acceso:** Creamos un `FutureProvider` para leer el disco (Data Source).
 4. **Lógica:** Creamos un `Notifier` para manipular los datos y manejar la inmutabilidad (State Management).
 5. **Presentación:** Usamos `ConsumerWidget` y `.when()` para mostrar los datos (UI).
-